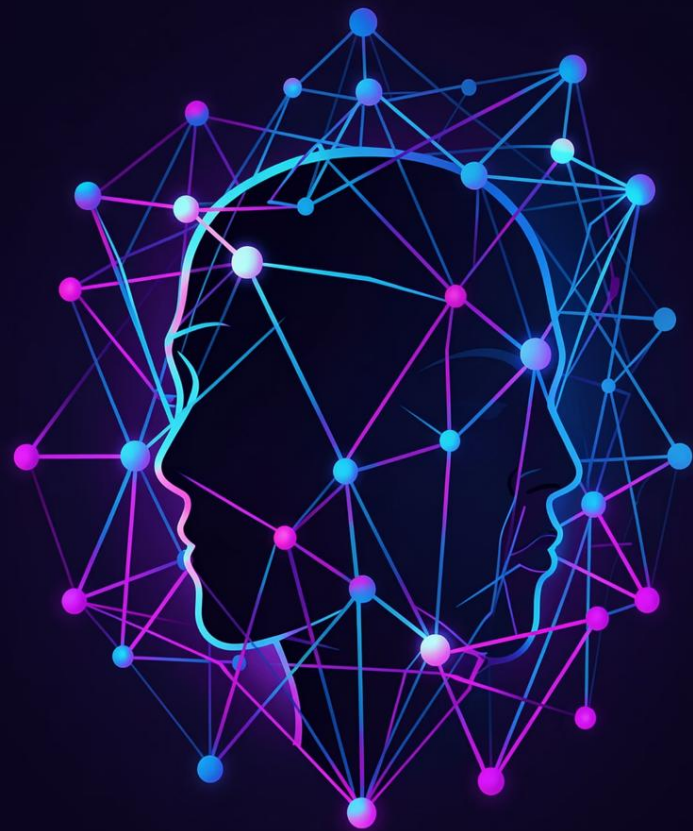




Sistema de Detección y Privacidad de Rostros

Automatizando la protección de menores en imágenes

Alejandro Jiménez Cabrera · Mayo 2026

¿De qué problema partimos?

Publicar fotos con menores sin anonimizarlos es un **riesgo real de privacidad**.
Revisar imágenes a mano no escala.

No escala

Con muchas imágenes, revisarlas una a una es inviable en tiempo y esfuerzo.

No es objetivo

Depende del criterio de cada persona, lo que genera inconsistencia.

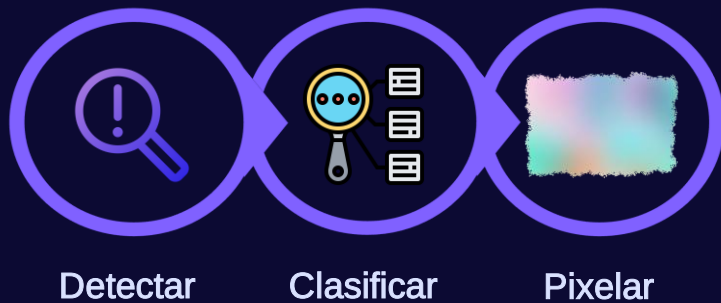
Es propenso a errores

Es fácil pasar por alto algún rostro, especialmente en fotos grupales.

- ❶ Nuestra solución: el usuario sube una foto y el sistema se encarga del resto. **Ninguna intervención humana.**



El resultado final, en tres pasos



El sistema devuelve tres versiones: la **original**, una con marcos **verdes** (adulto) o **rojos** (menor), y la **versión final** con los menores pixelados.



Original



Con marcos y resultado



Pixelada

Cómo está construido por dentro

El sistema se divide en **servicios independientes** que se comunican a través de [Apache Kafka](#), que actúa como tablón de anuncios: cada servicio publica su resultado y el siguiente lo recoge cuando está listo.

¿Qué pasa desde que subes la foto?

Siete etapas con trazabilidad completa: se registra la hora de inicio y fin de cada una.

Etapas	Servicio	Qué ocurre
Subida	API de entrada	Imagen guardada en MinIO y registrada en BD
Detección	Detection Service	RetinaFace localiza cada rostro
Orquestación	Orchestrator-2	Recorta cada cara y la guarda en MinIO
Clasificación	Age Service	Red neuronal puntúa: ≥ 0.45 = menor
Decisión	Orchestrator-3	Si no hay menores, cierra sin pixelar
Pixelado	Pixelation Service	Genera imágenes con marcos y pixelado
Consulta	API de resultados	El usuario accede al resultado con URLs

Cómo está construido por dentro (Casos)

El sistema puede actuar de manera distinta en el flujo dependiendo de 3 tipos de casos:

Caso 1 : No se detectan rostros

- El flujo se detiene en la fase de detección
- No se aplica clasificación ni pixelado
- Se registra sin rostros detectados

Caso 2 : Solo adultos detectados

- No se aplica pixelado
- La imagen se considera segura

Caso 3 : Menores detectados

- Se detectan rostros y al menos 1 es de menor
- Se cumple la traza complete
- Se generan las 3 imágenes finales

Tecnologías de soporte: Kafka (mensajería), MinIO (almacenamiento) y PostgreSQL (estado)

Docker conecta y despliega todos el servicios

Cómo detectamos si una cara es de un menor

Detección de rostros

RetinaFace (buffalo_l) localiza cada cara con alta precisión y devuelve sus coordenadas exactas.

Clasificación de edad

Red neuronal propia basada en **ResNet50**. Entrenamos solo la cabeza de clasificación con la base congelada.

Dataset desbalanceado

Aumento de datos (rotaciones, brillo, zoom) y mayor peso a los menores durante el entrenamiento.
3925 menores vs 5853 adultos

Umbral conservador

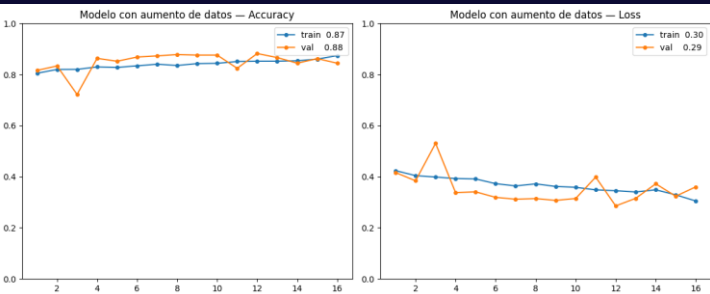
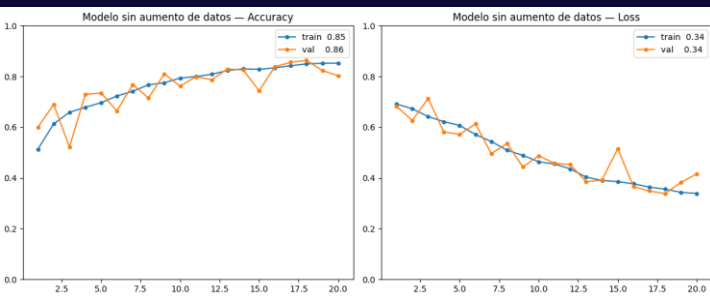
Puntuación ≥ 0.45 (no 0.50): preferimos pixelar un adulto de más a dejar un menor expuesto.

Dos decisiones que definen la calidad del sistema

¿Por qué ResNet50?

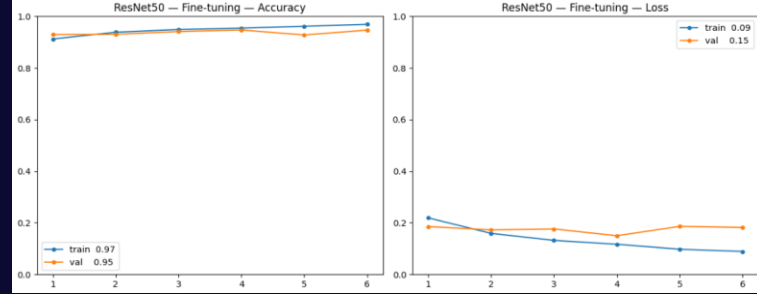
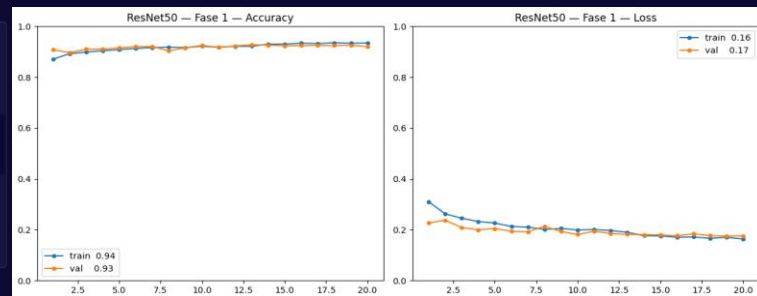
Pre-entrenada en 1,2M de imágenes: ya reconoce bordes y formas. Solo entrenamos la parte final.

Se han hecho pruebas comparando CNNs normales, CNNs con aumento de datos, Resnet50 con su base congelada y aumento de datos, y Resnet50 con FineTuning, y la conclusion ha sido que la major decision es Resnet con base congelada.



Modelo	Recall menores	Falsos negativos
CNN desde cero	84 %	~124
ResNet50 (base congelada)	95 %	42

Resnet congelado es la que proporciona valores más equilibrados de Accuracy y Loss y un mejor % de falsos negativos con solo 42.



Dos decisiones que definen la calidad del sistema

¿Por qué umbral de 0.45 y fijarse en el recall?

Con el umbral de 0.45 reducimos los falsos negativos a 42 casos, priorizando privacidad frente a 63 falsos positivos

Además, no todos los errores cuestan igual:

- **Falso positivo** (adulto → menor): molestia menor, sin consecuencias
- **Falso negativo** (menor → adulto): violación de privacidad

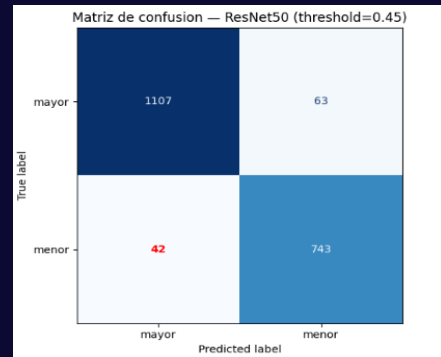
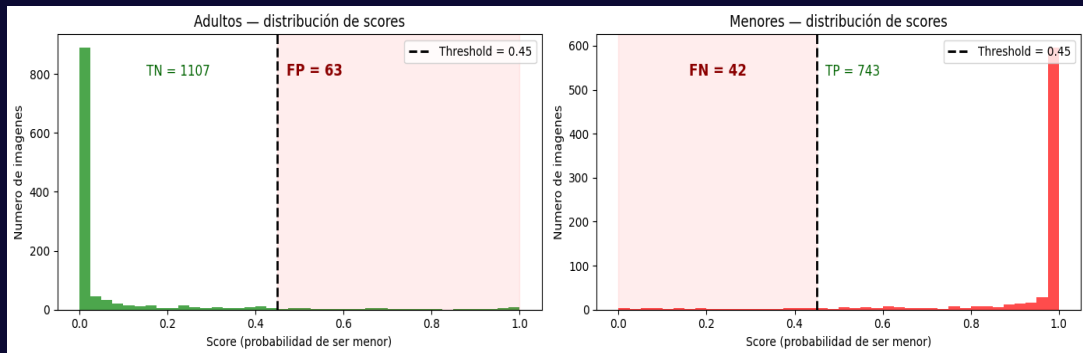
Recall (Sensibilidad)

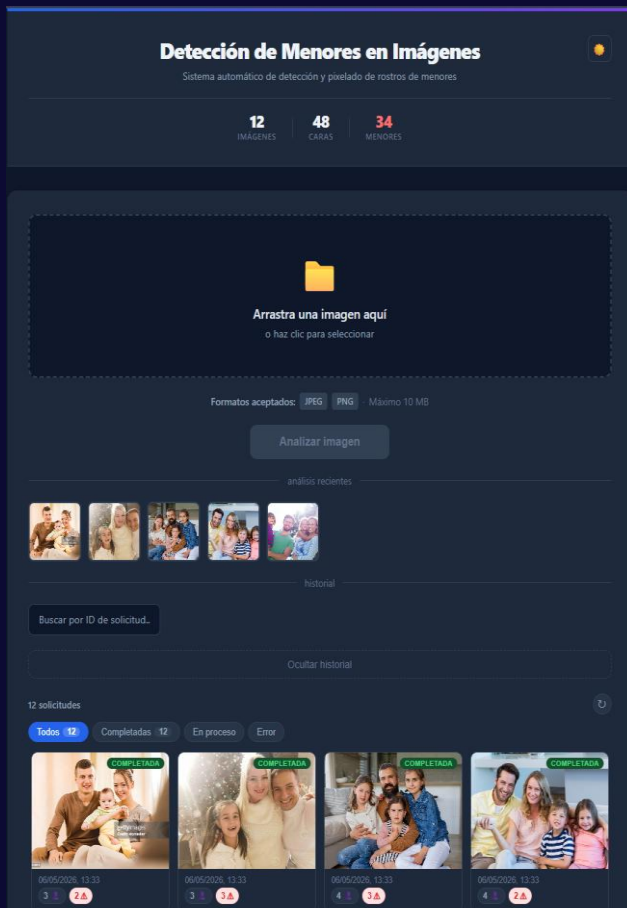
$$\text{Recall} = \frac{TP}{TP + FN} = \frac{743}{743 + 42} = \frac{743}{785} \approx 0.95$$

TP: Verdaderos positivos
(menores correctamente detectados)

FN: Falsos negativos
(menores no detectados)

Recall ≈ 0.95
Se detecta el 95% de los menores presentes





Lo que ve quién usa el sistema

Interfaz web en **React** con modo oscuro y validación robusta de ficheros.



Subida intuitiva

Arrastrar y soltar con previsualización. Validación leyendo imagen.



Progreso en tiempo real

Seguimiento del procesamiento etapa a etapa al finalizar.



Tres versiones del resultado

Original, con marcos y pixelada. Cada cara con su puntuación de confianza.



Historial y estadísticas

Miniaturas, filtros por estado, búsqueda por ID y métricas globales.

¿Qué pasa si algo falla?

El sistema está diseñado para **no perder trabajo** y recuperarse ante cualquier incidencia.

Mensajes Kafka con confirmación

No se confirman hasta que el procesamiento termina con éxito. Si un servicio cae, al reiniciar retoma desde donde lo dejó.

Nombres de fichero predecibles

Las imágenes se guardan con nombres fijos: reprocesar una solicitud no genera duplicados.

Errores irrecuperables visibles

Si un error es permanente (imagen corrupta), la solicitud se marca como ERROR y aparece en el historial.

Parada limpia de contenedores

Al apagar un contenedor, el sistema termina la tarea en curso antes de detenerse.

Conclusiones

Sistema funcional

Extremo a extremo, probado con imágenes reales de familias con menores y adultos mezclados.

Arquitectura event-driven

Cada servicio es intercambiable sin tocar el resto, ideal para pipelines de procesamiento.

Diseño alineado con el problema

Umbral 0.45 y el diseño de la red neuronal refleja decisiones conscientes.

Uso

Interfaz cómoda y de fácil acceso.

Despliegue en un comando

`docker compose up` arranca todo el sistema en cualquier máquina.

Propuestas de mejora

Respecto al modelo

Resnet50 con aumento de datos ya de por si funciona de manera más eficiente, pero quizás se puedan explorar otras arquitecturas que puedan darnos algún tipo de mejora.

El dataset tiene sesgo hacia adultos, añadir más imágenes de menores de distintas etnias y condiciones de luz mejoraría la generalización.

Respecto al sistema

Tener la posibilidad de poder escoger que modelo usar para detección de edad.

Si la base de datos o el almacenamiento falla, el sistema reintenta de inmediato, un reintento con espera progresiva sería más robusto ante cortes temporales.

Respecto al producto

Controlar quien puede subir y consultar imágenes.

FIN